

Language models

# Language models

- assign a probability
  - to each possible next word (token) in a sequence, e.g.,  $P(w_5 | w_1, w_2, w_3, w_4)$
  - to each sentence, e.g.,  $P(w_1, w_2, w_3, w_4, w_5)$
- useful, e.g., in spell checking, speech recognition, translation
- types
  - statistical (n-gram)
  - neural
  - transformer-based
- simpler approaches can be used for simple tasks, are fast, and introduce important concepts

# N-gram model

- the simplest model
- n is the number of words in a sequence
- we can compute (estimate) the probability of the next word using counting:

$$P(\text{of} \mid \text{we compute the probability}) = \frac{C(\text{we compute the probability of})}{C(\text{we compute the probability})}$$

- but
  - a large corpus is needed, too many possible sentences, we don't see enough examples
  - some sentences (sequences) do not occur

# N-gram model

- chain rule to calculate the joint probability can be used

$$P(w_1, w_2, \dots, w_n) = P(w_1) P(w_2 | w_1) P(w_3 | w_{1:2}) \dots P(w_n | w_{1:n-1})$$

- we can use not the entire sequence (history) but only a part of it = Markov assumption (a word depends only on the previous word/shorter history)
- for example, in a 3-gram model

$$P(w_1, w_2, \dots, w_5) = P(w_1) P(w_2 | w_1) P(w_3 | w_1 w_2) P(w_4 | w_2 w_3) P(w_5 | w_3 w_4)$$

- sometimes, a padding token can be added ( $P(w_1, w_2, \dots, w_5) = P(w_1 | <s><s>) P(w_2 | w_1 <s>) \dots$ ) so every word has the same history
- in a bigram model:

$$P(w_n | w_1, \dots, w_{n-1}) \approx P(w_n | w_{n-1})$$

- generally for an N-gram model:

$$P(w_n | w_1 \dots, w_{n-1}) \approx P(w_n | w_{n-(N-1)}, \dots, w_{n-1})$$

# N-gram model

- the probabilities are estimated (Maximum likelihood estimation) using counting:

$$P(w_n | w_{n-N+1}, \dots, w_{n-1}) = \frac{C(w_{n-N+1}, \dots, w_{n-1}, w_n)}{C(w_{n-N+1}, \dots, w_{n-1})}$$

# N-gram models

- instead of counting with probabilities, their logarithms are often used to prevent numerical underflow and to make computations faster, e.g.,
  - $p = p_1 * p_2 * p_3 * p_4$
  - $\log(p) = \log(p_1 * p_2 * p_3 * p_4) = \log p_1 + \log p_2 + \log p_3 + \log p_4$
  - $p = \exp(\log p_1 + \log p_2 + \log p_3 + \log p_4)$

# N-gram models evaluation

- we want the model predict words correctly, or assign high probabilities to real sentences
- must be evaluated on data the model has not seen (test set)
- probability of the test set decreases with the test set size (probabilities of n-grams are multiplied) => a metric normalized per word is better instead of counting with probabilities, their logarithms are often used, e.g.,
  - $\text{perplexity}(W = w_1 w_2 w_3 \dots w_N) = P(w_1 w_2 w_3 \dots w_N)^{-1/N} = \sqrt[N]{\frac{1}{P(w_1 w_2 w_3 \dots w_N)}}$
  - the higher the probability, the lower the perplexity (inverse in the formula)

# Perplexity

- perplexity is an intrinsic measure – does not depend on a specific task (e.g., spell checking)
- emphasizes the correctness of a probability distribution
- does not depend on a model architecture
- is a *weighted average branching factor* of a language
  - branching factor can be understood as the number of possible words that can follow a given word



# Perplexity – an example

- we are using a bigram model to predict the phrase: "Neural models"
- step 1: Predict "Neural" (given the start of the sentence)
  - the model is uncertain because many sentences could start differently
  - $P(\text{Neural} \mid \langle s \rangle) = 0.1$  (a branching factor of  $1/0.1 = 10$ )
- step 2: Predict "models" (given "Neural")
  - the model is very confident because "models" frequently follows "Neural"
  - $P(\text{models} \mid \text{Neural}) = 0.5$  (a branching factor of 2)
- perplexity =  $\sqrt[2]{\frac{1}{0.1 * 0.5}} = 4.47$  (geometric mean of branching factors)

# N-gram model

- n-gram models cannot handle long dependencies, e.g.,
  - The **actress** who won the award despite all the criticism from the media finally saw **herself** on the big screen.
  - The **manager** who supervises the three manufacturing plants in the northern territories **is** visiting tomorrow.
- n-gram models lack semantic generalization
  - words are completely unrelated (unlike word embeddings)
- problem with zero probabilities (something never occurred in training data)
  - can be addressed with smoothing or interpolation (using probabilities of sequences with shorter history)
  - neural approaches solve this with softmax or embeddings

# N-gram model

- neural architectures and LLMs dominate today as they address problems of n-gram models
- n-gram models are
  - simple, fast, require less resources
  - efficient for some tasks (e.g., spell checking, language identification, speech recognition)

# Neural language models

- more complex and powerful than N-gram models
- the inputs are usually represented by embeddings
  - this enables better predictions with unseen words, e.g.
    - context: "The chef prepared a salad with sliced pitaya and..."
    - n-gram: if "pitaya" was never in the training data, the N-gram model has no history
    - embedding: the embedding for "pitaya" is very close to "dragonfruit" and "mango", which is often followed by "lime" or "honey" – it predicts those correctly

# Neural language models

- a NN – takes some sequence of words  $w_1 \dots w_n$  as input and calculates the probability distribution  $y_i$  over possible next words
- the probabilities can be used in computing probabilities of sequences:

$$P(w_1, \dots, w_n) = \prod_{i=1}^n P(w_i | w_1, \dots, w_{i-1}) \Rightarrow \prod_{i=1}^n y_i[w_i]$$

$$P(w_{t+1} = k \mid w_1, \dots, w_t) = y_t[k]$$

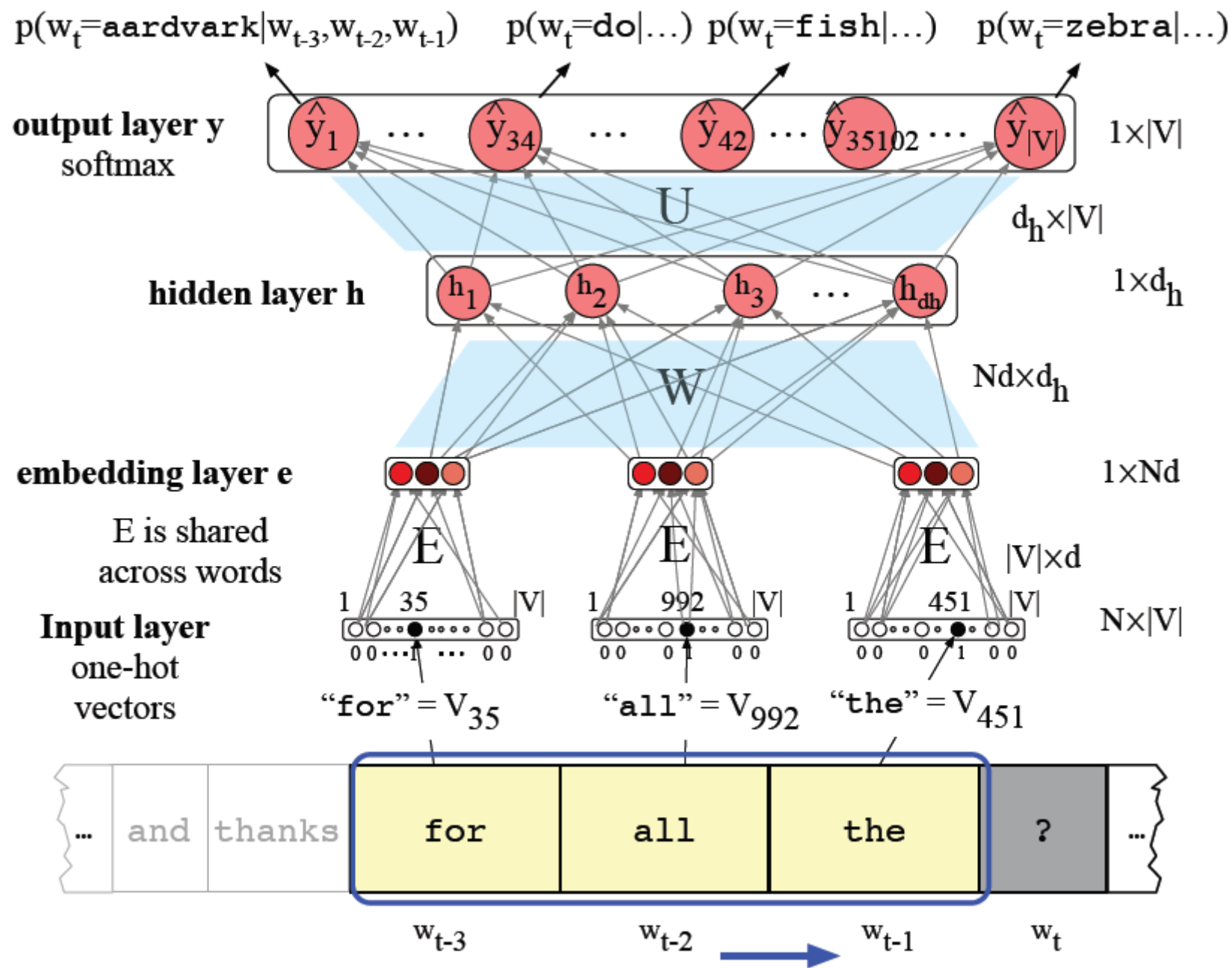
$y_t[k]$  ...  $k^{\text{th}}$  component of  $y$  (the output)

# Neural language models

- training
  - the task is to predict the next word
  - correct output = the next word = self-supervision algorithm (no labeling necessary)
  - the network tries to minimize the average loss for all predicted words, e.g., with GD

# Feedforward NN as LM

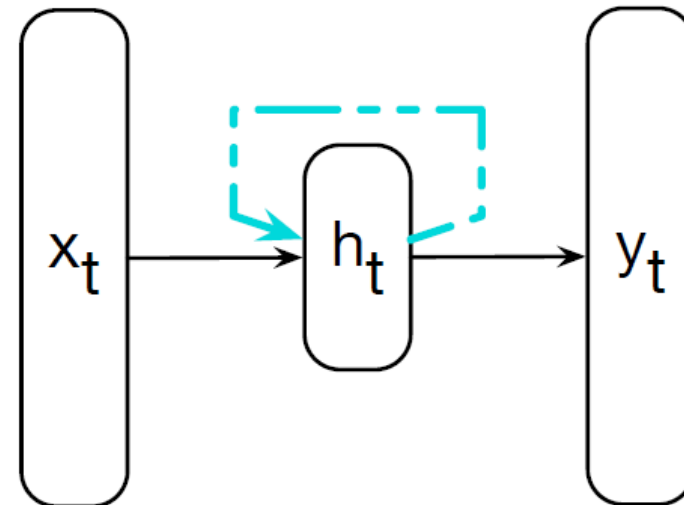
- input
  - some number of previous words (fixed length)
  - concatenating (not, e.g., averaging) embeddings preserves order
- output – probability distribution over the vocabulary





# RNNs as language models

- RNNs contain a loop – a hidden state serves as an input to the same layer in the next step of computation
- the current input is a single word, a sequence is processed in iterations => RNNs are able to work with long sequences, i.e., do not have a fixed context window size



# RNNs as language models

- RNN takes some sequence of words as input and calculates the probability distribution over possible next words
- the whole sequence of variable length is considered:

$$P(w_{1:n}) = \prod_{i=1}^n P(w_i | w_{<1:i-1}) \Rightarrow \prod_{i=1}^n y_i[w_i]$$

# RNNs as language models

- simple RNN models
  - an input is a word that uses an embedding matrix to create a part of the input to the hidden layer; it is combined with the previous output from the hidden layer
  - the output from the hidden layer goes to a softmax layer (it also has weights)

$$e_t = Ex_t$$

$$h_t = g(Uh_{t-1} + We_t)$$

$$y_t = \text{softmax}(Vh_t)$$

$U, V, W$  ... weight matrices

$E$  ... embedding matrix

$x_t$  ... input

# RNNs as language models

- the embedding matrix  $E$  and the weight matrix  $V$  for the softmax layer have a similar purpose, to represent relevant aspects of words:
  - $E$  ... represents input embeddings, similar words should have similar embeddings; the dimension is  $d_h \times |V|$  ( $d_h$  ... the number of hidden units)
  - $V$  ... represents the weights for words in the output layer; dimensionality  $|V| \times d_h$
- only one matrix can be used (=weight tying) => less parameters of the model

$$y_t = \text{softmax}(E^T h_t)$$

# RNN variants

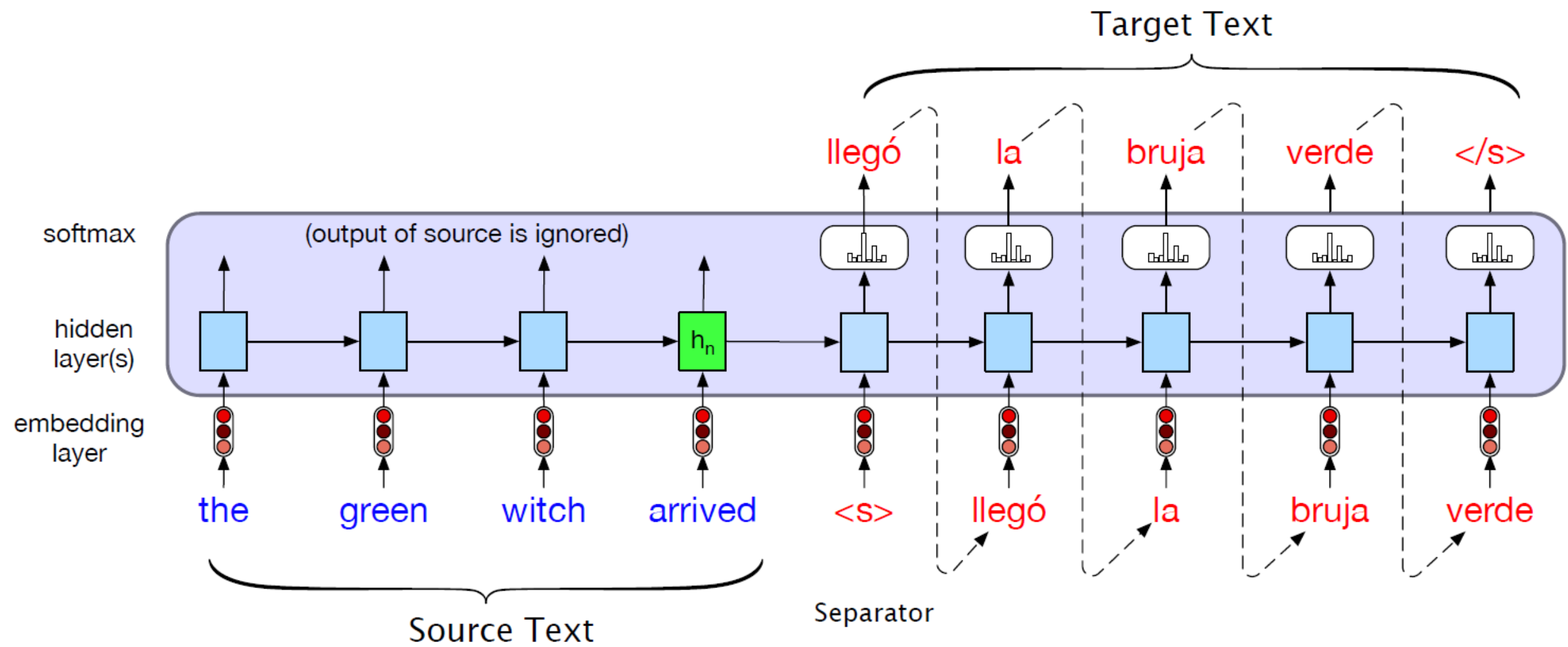
- stacked RNNs
  - first layer takes  $x_1, x_2, \dots$  as the input sequence and produces outputs
  - these outputs are the input to the second layer etc.
  - the last layer produces the final outputs  $y_1, y_2, \dots$  of the model
- bidirectional RNNs
  - two independent RNNs that process the sequence from start to end and from end to start
  - their outputs are aggregated
  - only the final outputs of both NN can be used, e.g., in classification
- gated NN
  - RNNs suffer from the vanishing gradient problem
  - LSTM and GRU gates are often used

# Sequence-to-sequence models (encoder-decoder)

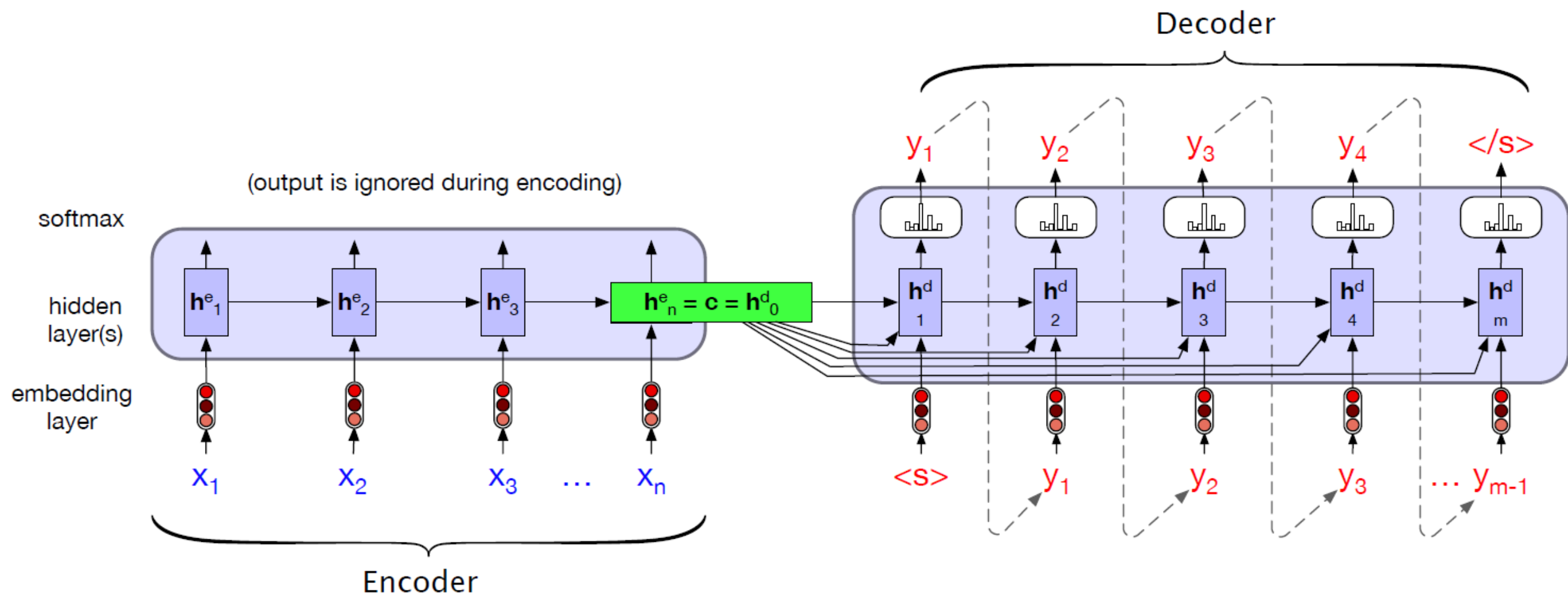
- for generating a sequence based on another with a different length
- encoder
  - encodes a sequence of inputs into a context vector (the output of the hidden layer)
  - the outputs  $y$  are ignored
  - the context vector serves as the initial input for a decoder

# Sequence-to-sequence models (encoder-decoder)

- decoder
  - generates a new sequence based on the context (prior hidden state) and the separating symbol (end/start of a sentence) until an end-of-sequence token is generated
  - instead of the previously generated output, the real output can be used as input during training = teacher forcing
  - the context can also be available for all steps in the decoder
- encoder and decoder can be of any kind of sequence architecture
- stacked encoders (e.g., stacked bidirectional LSTM) are common







# Problems of seq2seq models

- in a vanilla (=basic) encoder-decoder model, the decoder can access only the last hidden state of the encoder (=bottleneck)
- the last state
  - no order of the included information is preserved
  - must aggregate sometimes a very long sequence
  - is mostly influenced by the last encoder states

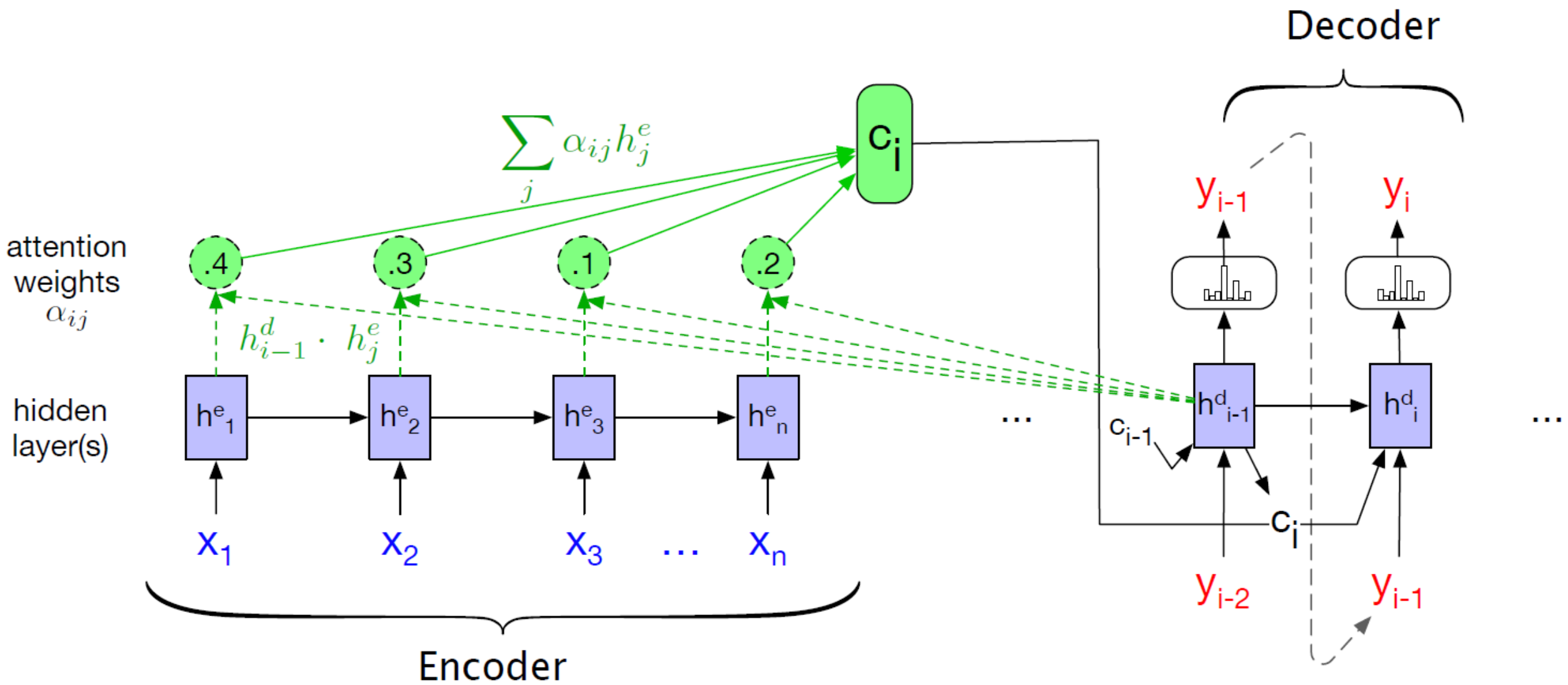
# Attention

- enables accessing all hidden states of the encoder by the decoder
  - instead of accessing the context vector, which is the encoder's final hidden state, a context vector  $c_i$  for the  $i$ th decoder step, is composed of all encoder's hidden states:  $c_i = f(h_1^e \dots h_n^e, h_{i-1}^d)$
  - the number of hidden states varies (input sequences have different lengths)  
=> a single vector of a fixed length needs to be provided to the decoder at each step => weighted sum of all the state vectors is calculated
  - the weights represent the importance of (pay attention to) a part of the source with respect to the output being generated = the context vector  $c_i$  is dynamic, different for each token in decoding

$$h_i^d = g(\hat{y}_{i-1}, h_{i-1}^d, c_i)$$

# Attention

- to compute  $c_i$ , the importance  $score(h_{i-1}^d, h_j^e)$  of each encoder state  $h_j^e$  for the previous decoder state  $h_{i-1}^d$  must be calculated
- dot-product attention
  - the simplest form, measures similarity between two states
  - $score(h_{i-1}^d, h_j^e) = h_{i-1}^d \cdot h_j^e$
  - the dot product is normalized using softmax over all  $j$
  - the normalized scores are used as weights for the weighted sum



# Attention

- the different aspects of similarity can be computed in a more sophisticated way
- Luong attention (bilinear attention)
  - parametrized using weights (linearly transform the encoder states) that are learned

$$score(h_{i-1}^d, h_j^e) = h_{i-1}^d \cdot W_S h_j^e$$

- Bahdanau attention (additive, MLP attention)
    - the vectors are concatenated and passed through a simple NN
- $$score(h_{i-1}^d, h_j^e) = v^T \tanh(W[h_{i-1}^d; h_j^e]) \text{ or }$$
- $$score(h_{i-1}^d, h_j^e) = v^T \tanh(W_1 h_{i-1}^d + W_2 h_j^e)$$
- can learn complex relationships, but is slow